임베딩과 벡터 저장소 (실습)

3-1. 임베딩의 개념
3-2. 유사도 검색 체험 (실습)
3-3. 벡터 저장소 구축 (실습)
3-4. 쉬어가기: 임베딩은 어떻게 '의미'를 갖게 됐나

3-1. 임베딩의 개념

> 문장을 벡터로 바꾼다는 것이 무엇을 뜻하는지 눈으로 확인해봅니다.

임베딩 모델은 텍스트를 받아 고정 길이 실수 벡터(소수점 숫자를 정해진 개수만큼 늘어놓은 배열)로 바꿉니다. 이번 교육에서 쓰는 Amazon Titan Embed Text v2는 어떤 길이의 문장이든 1024개의 실수로 바꿉니다.

의미가 비슷한 문장은 가까운 벡터가 됩니다. "연차 며칠 받아?"와 "월차 얼마나 돼?"는 단어가 거의 안 겹치지만 벡터 공간에서는 가까이 놓입니다.

키워드 검색과 비교하면 차이가 드러납니다.

```
사용자 질문: "월차 며칠?"
문서 내용:  "연차 휴가는 연 15일이 부여됩니다"

키워드 검색: 실패 ("월차"라는 단어가 문서에 없음)
임베딩 검색: 성공 (의미가 가까우므로 벡터 거리가 가까움)
```

임베딩을 쓰면 단어가 아니라 뜻으로 검색할 수 있습니다.

따라하기

1. 아래 코드를 `embedding.py` 로 저장하고 실행해 문장 하나를 임베딩한 벡터의 실체를 봅니다.

```
# embedding.py — 임베딩 확인
from dotenv import load_dotenv
from langchain_aws import BedrockEmbeddings

load_dotenv()
embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1",
)

v = embeddings.embed_query("연차 휴가는 며칠인가요?")
print(f"벡터 차원: {len(v)}")
print(f"앞 5개 값: {[round(x, 4) for x in v[:5]]}")
```

2. 출력을 확인합니다. 1024개의 숫자 자체에 사람이 읽을 수 있는 의미는 없습니다. 의미는 벡터들 사이의 거리 관계에만 있습니다.

막힐 때

- `AccessDeniedException` 이 납니다.

  임베딩 모델은 채팅 모델과 별개로 권한이 필요합니다. model_id 오타와 region 설정부터 확인하고 그래도 나면 자격 증명에 임베딩 모델 권한이 포함됐는지 문의합니다.

확인: 벡터 차원 1024가 출력되면 성공입니다.

> 임베딩이 문장을 고정 길이 벡터로 바꾸고 의미가 비슷하면 벡터가 가깝다는 개념을 익혔습니다.

3-2. 유사도 검색 체험 (실습)

> 코사인 유사도로 문장 쌍의 의미가 얼마나 가까운지 직접 재봅니다.

두 벡터가 얼마나 가까운지 재는 표준 자는 코사인 유사도입니다. 두 벡터 사이 각도의 코사인 값으로 1에 가까울수록 의미가 비슷하고 0에 가까울수록 무관합니다.

따라하기

1. 아래 코드를 `similarity.py` 로 저장하고 실행해 네 가지 문장 쌍의 유사도를 비교합니다.

```
# similarity.py - 코사인 유사도 체험
import math
from dotenv import load_dotenv
from langchain_aws import BedrockEmbeddings

load_dotenv()
embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1",
)


def cosine(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    na = math.sqrt(sum(x * x for x in a))
    nb = math.sqrt(sum(x * x for x in b))
    return dot / (na * nb)


pairs = [
    ("연차 며칠 받을 수 있어?", "월차 휴가 일수 알려줘"),   # 같은 의미, 다른 단어
    ("연차 며칠 받을 수 있어?", "VPN 설정 방법"),           # 무관한 주제
    ("Python", "파이썬"),                                    # 다른 언어, 같은 의
미
    ("출장비 정산", "경비 처리 기한"),                       # 비슷한 업무 영역
]

for a, b in pairs:
    va, vb = embeddings.embed_query(a), embeddings.embed_query(b)
    print(f"'{a}' vs '{b}' -> {cosine(va, vb):.3f}")
```

   코드의 `sum(x * y for x, y in zip(a, b))` 는 두 벡터의 같은 자리끼리 곱해 모두 더하는 내적 계산입니다. `zip(a, b)` 가 두 리스트를 앞에서부터 짝지어 주고, 괄호 안의 `... for ... in ...` 이 그 짝을 돌면서 값을 하나씩 만들어 냅니다. Java 스트림과 성격이 비슷합니다.

2. 출력된 네 쌍의 점수를 비교합니다.
   - 단어가 하나도 안 겹쳐도 의미가 같으면 높은 점수가 나옵니다.
   - 영어 Python과 한글 파이썬도 가깝습니다. 임베딩 모델이 여러 언어를 같은 벡터 공간에 놓기 때문입니다.
   - 점수의 절대값보다 후보들 사이의 순위가 중요합니다. 0.7 이상이면 관련 있다는 식의 고정 기준은 모델을 교체하는 순간 쓸모없어집니다.

막힐 때

- `embed_query` 와 `embed_documents` 가 헷갈립니다.

  질문 하나는 `embed_query(str)` 를 쓰고 문서 여러 개는 `embed_documents(list[str])` 를 씁니다. 일부 모델은 질문용과 문서용 변환이 실제로 다르므로 용도에 맞는 함수를 씁니다.

- 호출이 느리고 비용이 걱정됩니다.

  임베딩도 API 호출이라 문장마다 과금됩니다. 실험 중에는 결과를 변수에 담아 재사용합니다.

확인: 같은 의미 쌍의 점수가 무관한 주제 쌍보다 뚜렷하게 높게 나오면 성공입니다.

> 의미 유사도가 벡터 거리로 계산된다는 걸 직접 재보며 확인했습니다.

3-3. 벡터 저장소 구축 (실습)

> 사내 정책 문서 청크를 Chroma 벡터 DB에 적재하고 디스크에 저장해봅니다.

청크가 수천 개면 질문이 올 때마다 전부와 코사인 유사도를 계산할 수 없습니다. 벡터 저장소는 청크와 임베딩을 저장해 두고 질문 벡터와 가장 가까운 청크를 빠르게 찾아주는 전용 DB입니다.

교육에서는 Chroma를 씁니다. 설치가 pip 한 줄이고 서버 없이 로컬 파일로 동작하기 때문입니다. 실무에서는 Pinecone, pgvector, Qdrant, OpenSearch 등을 쓰지만 인터페이스 개념은 같습니다.

`persist_directory` 를 지정하지 않으면 벡터가 메모리에만 있다가 프로그램 종료와 함께 사라지고 실행할 때마다 임베딩 비용을 다시 냅니다. 지정하면 벡터가 폴더에 저장되어 다음 실행부터는 불러오기만 하면 됩니다.

오늘 만드는 `chroma_db` 폴더는 6일차 통합 실습에서 그대로 다시 사용합니다. 3일차 선택 실습에서도 쓰므로 오늘 실습이 끝나도 지우지 않습니다.
`collection_name="sds_policies"` 와 `persist_directory="./chroma_db"` 는 전 과정에서 고정된 이름이니 바꾸지 않습니다.

따라하기

1. 아래 코드를 `vectorstore.py`로 저장하고 실행해 2번째 세션 2-3에서 만든 청크를 적재한 뒤 의미 검색과 필터 검색을 확인합니다.

```
# vectorstore.py — 벡터 저장소 구축과 디스크 저장
from dotenv import load_dotenv
from langchain_aws import BedrockEmbeddings
from langchain_chroma import Chroma
from metadata import load_chunks

load_dotenv()
embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1",
)

db = Chroma.from_documents(
    documents=load_chunks(),
    embedding=embeddings,
    collection_name="sds_policies",
    persist_directory="./chroma_db",   # 이 폴더에 저장됩니다 (6일차까지 사용)
)
print(f"적재 완료: {db._collection.count()}개 청크")

# 의미 검색 확인 (점수 포함)
for q in ["월차 며칠 받을 수 있어?", "집에서 일해도 되나요?", "법카 등록 기한"]:
    hits = db.similarity_search_with_score(q, k=2)
    print(f"\n질문: {q}")
    for doc, score in hits:
        print(f"  [{doc.metadata['source']}] (거리 {score:.3f}) {doc.page_con
tent[:40]}...")

# 메타데이터 필터 검색
hits = db.similarity_search("경비 정산 규정", k=2, filter={"department": "총무
팀"})
print(f"\n총무팀 문서로 한정 검색: {[h.metadata['source'] for h in hits]}")
```

   `db._collection.count()` 처럼 이름이 밑줄로 시작하는 것은 파이썬에서 바깥에서 갖다 쓰라고 만든 것이 아니라는 표시입니다. 여기서는 적재된 개수를 확인하려고 잠깐 들여다볼 뿐이고, 실제 검색은 아래의 `similarity_search` 같은 공개 메서드로 합니다.

2. 출력을 관찰합니다. 월차와 법카처럼 문서에 없는 단어로도 올바른 청크가 찾아집니다.
   `similarity_search_with_score`의 점수는 거리라서 작을수록 가깝습니다. 코사인 유사도와 방향이 반대입니다.

3. 저장된 DB를 다시 불러올 때는 `from_documents`가 아니라 생성자를 씁니다. 아래 코드를 `load_db.py`로 저장해 둡니다. 뒤 실습에서 문제가 생겼을 때 청크 수 확인용으로 다시 씁니다.

```
# load_db.py - 두 번째 실행부터는 이렇게 (임베딩 비용 0)
from dotenv import load_dotenv
from langchain_aws import BedrockEmbeddings
from langchain_chroma import Chroma

load_dotenv()
embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1",
)

db = Chroma(
    collection_name="sds_policies",
    embedding_function=embeddings,     # 질문을 임베딩할 때 필요하므로 여전히 지정
    persist_directory="./chroma_db",
)
print(f"불러온 청크 수: {db._collection.count()}")
```

막힐 때

- 재실행할 때마다 청크 수가 늘어납니다.

  `from_documents`를 다시 실행하면 같은 내용이 또 들어갑니다. 다시 적재하기 전에 `chroma_db` 폴더를 지우거나 두 번째 실행부터는 불러오기 코드를 씁니다. 실무에서는 청크마다 고유 `ids`를 지정해 덮어쓰는 방식을 씁니다.

- Chroma import 오류가 납니다.

  `langchain_community.vectorstores.Chroma`는 구버전 경로입니다. `pip install langchain-chroma` 후 `from langchain_chroma import Chroma`를 씁니다.

- 불러왔는데 청크 수가 0입니다.

  `persist_directory`는 같은데 `collection_name`이 다르면 빈 컬렉션이 새로 만들어집니다. 두 값을 모두 일치시킵니다.

- 임베딩 모델을 중간에 바꾸면 안 됩니다.

  적재할 때 쓴 모델과 검색할 때 쓰는 모델이 다르면 벡터 공간이 달라 검색이 엉망이 됩니다. 모델을 바꾸면 전체 재적재가 필요합니다.

확인: 적재된 청크 수가 출력되고 문서에 없는 단어로 질문해도 올바른 파일의 청크가 검색되면 성공입니다.

벡터 DB를 디스크에 저장해 두면 재실행 시 임베딩 비용 없이 불러올 수 있다는 걸 확인했습니다.

3-4. 쉬어가기: 임베딩은 어떻게 '의미'를 갖게 됐나

벡터가 왜 의미를 담게 됐는지, 코사인과 1024차원은 어디서 나온 선택인지 짚어봅니다.

시작은 2013년 word2vec입니다. 단어를 벡터로 바꿔 놓고 king - man + woman을 계산했더니 결과가 queen 근처에 떨어졌습니다. 의미를 산술로 다룰 수 있다는 걸 처음 눈으로 본 순간이었습니다.

문제는 단어 단위라 문맥을 못 본다는 것이었습니다. "은행에 다녀왔다"의 은행이 금융기관인지 강둑인지 구분하지 못했습니다.
2018년 BERT 이후 문맥을 반영한 문장 임베딩으로 넘어갔고, 오늘 쓴 Titan Embed Text v2도 이 계열입니다.

코사인 유사도를 쓰는 이유도 여기서 나옵니다.

- 코사인은 방향만 봅니다. 문장이 길어 벡터가 커져도 의미가 같으면 점수가 유지됩니다.
- 유클리드 거리를 쓰면 긴 문장이 구조적으로 불리합니다. 검색에서 짧은 청크만 계속 뽑히는 현상이 여기서 옵니다.

차원 수는 정답이 없습니다. 384, 768, 1024, 1536을 흔히 쓰는데 낮으면 의미가 뭉개지고 높으면 저장 용량과 검색 비용이 같이 올라갑니다.
Titan v2가 1024를 기본값으로 잡은 것도 품질과 비용을 반씩 양보한 결과입니다.

3-3에서 임베딩 모델을 바꾸면 전부 다시 넣어야 한다고 한 이유가 이것입니다. 차원 수가 같아도 좌표계가 다른 모델이라 기존 벡터와 비교 자체가 성립하지 않습니다.

벡터의 방향이 곧 의미이고, 코사인과 차원 수는 그 위에서 고른 절충이라는 걸 확인했습니다.